PA4

CSE 3150 Fall 2025 - Song Han

Due 10/24/2025, 11:59pm.

---

1 Matrix Implementation

For this assignment, you will implement a matrix data structure. A header file has been provided for you with all of the necessary function declarations.
Your job is to fill in the .cpp file. Use the comments in the code to guide you. The following are some descriptions for the functions. You should implement the matrix using a 2D vector (not regular arrays which cannot be resized). This should make it easier for you, as memory is automatically managed by the STL container. Some matrix operations are undefined for matrices of different sizes. Make sure you check for these scenarios and return a 1x1 matrix, {0}, if you cannot perform the calculation.

The following is a description of each function you need to implement. We suggest you implement the constructors and display() function first to aid in debugging as you write the rest.

1. Default Constructor: Matrix(size_t r, size_t c); - Create a new matrix object, with r rows and c columns.
2. Copy Constructor: Matrix(const Matrix& original); - Make an exact copy of the matrix passed in!
3. Operator Overloads:
   (a) Equality: bool operator==(const Matrix& rhs) const; - Assess equality of two matrices, both by size/dimensions and cell contents.
   (b) Addition: Matrix operator+(const Matrix& rhs) const; - Add the corresponding cells of this matrix with the right hand side, returning the resulting matrix.
   (c) Subtraction: Matrix operator-(const Matrix& rhs) const; - Subtract corresponding cells between matrices and return the result.
   (d) Multiplication: Matrix operator*(const Matrix& rhs) const; - Perform Matrix Multiplication and return the resulting matrix.
   (e) Getter/Setter: T& operator()(size_t r, size_t c); - Setter and Getter for non-const matrices. Because you cannot override [][] in C++, we use this one to get around it.
   (f) Getter: const T& operator()(size_t r, size_t c) const; - Getter only; very similar to the previous one, except it allows values to be read from const matrices.
4. Other Utility Functions:
   (a) Transpose: Matrix transpose() const; - Transpose a matrix, turning the columns of the original matrix into the rows of the matrix after the operation.
   (b) Get Number of Rows: size_t getNumRows(); - Simply return the number of rows in a matrix.
   (c) Get Number of Cols: size_t getNumCols(); - Simply return the number of columns in a matrix.
   (d) Display: void display() const; - Returns nothing, but prints a visual of your matrix to the console.